

# declcall(unevaluated-postfix-expression)

Document #: P2825R1  
Date: 2024-03-21  
Project: Programming Language C++  
Audience: EWG  
Reply-to: Gašper Ažman  
<[gasper.azman@gmail.com](mailto:gasper.azman@gmail.com)>

## Contents

- [1 Introduction](#)
- [2 Motivation and Prior Art](#)
  - [2.1 Related Work](#)
- [3 Proposal](#)
  - [3.1 Interesting cases in the above example](#)
  - [3.2 Alternatives to syntax](#)
  - [3.3 Naming](#)
- [4 Usecases](#)
  - [4.1 What does this give us that we don't have yet](#)
  - [4.2 That's not good enough to do all that work. What else?](#)
- [5 Needed Guidance](#)
- [6 References](#)

## § 1 Introduction

This paper introduces a new compile-time expression into the language, for the moment with the syntax `declcall(postfix-expression)`.

The expression is a constant expression of the type pointer-to-function (PF) or pointer-to-member-function (PMF). Its value is the pointer to the function that would have been invoked if the `[_postfix-expression_]@` were evaluated. The `[_postfix-expression_]@` itself is an unevaluated operand.

In effect, `declcall` is a hook into the overload resolution machinery.

## § 2 Motivation and Prior Art

The language already has a number of sort-of overload resolution facilities:

- `static_cast`
- assignment to a variable of a given function pointer type
- function calls (implicit) - the only one that actually works

All of these are woefully unsuitable for type-erasure that library authors (such as [P2300R6]) would actually *like* to work with. Sure, we can always indirect through a lambda:

```
template <typename R, typename Args...>
struct my_erased_wrapper {
    using fptr = R (*)(Args...);
    fptr erased = +[](my_erased_wrapper* self, auto&&... args_) -> R {
        return self->fptr(std::forward<decltype>(args_)...);
    };
};
```

This has several drawbacks:

- it introduces a whole new lambda scope
  - annoying for optimizers because extra inlining
  - annoying for debugging because of an extra stack frame
- a requirement to divine forward noexceptness by the user,
- a requirement that arguments are *convertible* (but not necessarily the same) as the ones the user specified in their infinite wisdom,
- a requirement that the return type is convertible (but not necessarily the same) as the one the user specified in their infinite wisdom,
- the forward-through-a-reference inhibits copy-elision of arguments
- inability to flatten subsetting type-erased wrappers (we can't divine an exact match in the presence of overloads) (think “subsetting vtables”)

Oh, if only we had a facility to ask the compiler what function we'd be calling and then *just have the pointer to it*.

This is what this paper is trying to provide.

### § 2.1 Related Work

#### § 2.1.1 Reflection

The reflection proposal includes `reflect_invoke`, which would allow one to do the same thing, but with more work to get the actual pointer. We probably need to do the

specification work of this paper to understand the corner cases of `reflect_invoke`.

However, reflection ([P2320R0],[P1240R1],[P2237R0],[P2087R0],[N4856]) is might miss C++26, and is far wider in scope as another `decltype`-ish proposal that's easily implementable today, and `std::execution` could use immediately.

Regardless of how we chose to provide this facility, it is dearly needed, and should be provided by the standard library or a built-in.

See the [Alternatives to Syntax](#) chapter for details.

## § 2.1.2 Library fundamentals TS v3

The [Library Fundamentals TS version 3](#) defines `invocation_type<F(Args...)>` and `raw_invocation_type<F(Args...)>` with the hope of getting the function pointer type of a given call expression.

However, this is not good enough to actually be able to resolve that call in all cases.

Observe:

```
struct S {
    static void f(S) {} // #1
    void f(this S) {}   // #2
};
void h() {
    static_cast<void(*)(>S::f) // error, ambiguous
    S{}.f(S{}); // calls #1
    S{}.f();    // calls #2
    // no ambiguity for declcall
    declcall(S{}.f(S{})); // &#1
    declcall(S{}.f());    // &#2
}
```

A library solution can't give us this, no matter how much we try, unless we can reflect on unevaluated operands (which Reflection does).

## § 3 Proposal

We propose a new (technically) non-overloadable operator (because `sizeof` is one, and this behaves similarly):

```
declcall(postfix-expression);
```

Example:

```
int f(int); // 1
int f(long); // 2
constexpr auto fptr_to_1 = declcall(f(2));
constexpr auto fptr_to_2 = declcall(f(2L));
```

The program is ill-formed if the named *postfix-expression* is not a call to an addressable function (such as a constructor, destructor, built-in, etc.).

```
struct S {};  
declcall(S()); // Error, constructors are not addressable  
declcall(__builtin_unreachable()); // Error, not addressable
```

The expression is not a constant expression if the *postfix-expression* does not resolve for unevaluated operands. Examples of this function pointer values and surrogate functions.

```
int f(int);  
using fptr_t = int (*)(int);  
constexpr fptr_t fptr = declcall(f(2));  
declcall(fptr(2)); // Error, fptr_to_1 is a pointer value  
struct T {  
    constexpr operator fptr_t() const { return fptr; }  
};  
declcall(T{}(2)); // Error, T{} would need to be evaluated
```

If the `declcall(postfix-expression)` is evaluated and not a constant expression, the program is ill-formed (but SFINAE-friendly).

However, if it is unevaluated, it's not an error.

Example:

```
int f(int);  
using fptr_t = int (*)(int);  
constexpr fptr_t fptr = declcall(f(2));  
static_cast<declcall(fptr(2))>(fptr); // OK, fptr, though redundant  
struct T {  
    constexpr operator fptr_t() const { return fptr; }  
};  
static_cast<declcall(T{}(2))>(T{}); // OK, fptr
```

This pattern covers all cases that need evaluated base operands, while making it explicit that the operand is evaluated due to the static cast.

Examples:

```
void g(long x) { return x+1; }  
void f() {} // #1  
void f(int) {} // #2  
struct S {  
    friend auto operator+(S, S) noexcept -> S { return {}; } // #3  
    auto operator-(S) -> S { return {}; } // #4  
    auto operator-(S, S) -> S { return {}; } // #5  
    void f() {} // #6  
    void f(int) {} // #7  
    S() noexcept {} // #8  
    ~S() noexcept {} // #9  
    auto operator->(this auto&& self) const -> S*; // #10  
    auto operator[](this auto&& self, int i) -> int; // #11
```

```

static auto f(S) -> int; // #12
using fptr = void(*) (long);
auto operator fptr const { return &g; } // #13
auto operator<=>(S const&) = default; // #14
};
S f(int, long) { return S{}; } // #15
struct U : S {}

void h() {
    S s;
    U u;
    declcall(f()); // ok, &#1 (A)
    declcall(f(1)); // ok, &#2 (B)
    declcall(f(std::declval<int>())); // ok, &#2 (C)
    declcall(f(1s)); // ok, &#2 (!) (D)
    declcall(s + s); // ok, &#3 (E)
    declcall(-s); // ok, &#4 (F)
    declcall(-u); // ok, &#4 (!) (G)
    declcall(s - s); // ok, &#5 (H)
    declcall(s.f()); // ok, &#6 (I)
    declcall(u.f()); // ok, &#6 (!) (J)
    declcall(s.f(2)); // ok, &#7 (K)
    declcall(s); // error, constructor (L)
    declcall(s.S::~~S()); // error, destructor (M)
    declcall(s->f()); // ok, &#6 (not &#10) (N)
    declcall(s.S::operator->()); // ok, &#10 (O)
    declcall(s[1]); // ok, &#11 (P)
    declcall(S::f(S{})); // ok, &#12 (Q)
    declcall(s.f(S{})); // ok, &#12 (R)
    declcall(s(1l)); // error, #13 (S)
    static_cast<declcall(s(1l)>(s)); // ok, &13 (S)
    declcall(f(1, 2)); // ok, &#15 (T)
    declcall(new (nullptr) S()); // error, not function (U)
    declcall(delete &s); // error, not function (V)
    declcall(1 + 1); // error, built-in (W)
    declcall([]{
        return declcall(f());
    })(); // ok, &2 (X)
    declcall(S{} < S{}); // error, synthesized (Y)
}

```

## § 3.1 Interesting cases in the above example

- resolving different members of a free-function overload set (A, B, C, D, T)
  - the (D) case is important - the short argument still resolves to the int overload!
- constructors and destructors (L, M, U, V) - see the **possible extensions** chapter.
- resolving different member of a member-function overload set (I, J, K, N, Q, R)
  - the (J) example is important - the call on u still resolves to a member function of S.

- resolving built-in non-functions (W): we could make this work in a future extension (see that chapter).
- resolving operator `>` (N and O). specifies that *postfix-expressions* group left-to-right, which means the top-most postfix-expression is the call to `f()`, and not the `->`. To get to `S::operator->`, we have to ask for it explicitly.
- surrogate function call (S) - again, the top-most call-expression is the function call to `g`, so the type of `g` is returned, but it's not a constant expression. We can get it by evaluating the operand with `static_cast`.
- nested calls: (X) the top-level call is a call to a function-pointer to #2, so that is what is returned.
- Synthesized operators (Y) - these are not functions that we can take pointers to, so unless we “force-manufacture” one, we can't make this work.

## § 3.2 Alternatives to syntax

We could wait for reflection in which case `declcall` is implementable when we have expression reflections.

```
namespace std::meta {
    template<info r> constexpr auto declcall = []{
        if constexpr (is_nonstatic_member(r)) {
            return pointer_to_member<[:pm_type_of(r):]>(r);
        } else {
            return entity_ref<[:type_of:]>(r);
        } /* insert additional cases as we define them. */
    }();
}

int f(int); //1
int f(long); //2
constexpr auto fptr_1 = [: declcall<^f(1)> :]; // 1
```

It's unlikely to be quite as efficient as just hooking directly into the resolver, but it does have the nice property that it doesn't take up a whole keyword.

Many thanks to Daveed Vandevoorde for helping out with this example.

## § 3.3 Naming

I think `declcall` is a reasonable name - it hints that it's an unevaluated operand, and it's how I implemented it in clang.

For all intents and purposes, this facility grammatically behaves in the same way as `sizeof`, except that we should require the parentheses around the operand.

We could call it something really long and unlikely to conflict, like

- `declcall`

- declinvoke
- calltarget
- expression\_targetof
- calltargetof
- decltargetof
- resolvetarget

## § 4 Usecases

Broadly, anywhere where we want to type-erase a call-expression. Broad uses in any type-erasure library, smart pointers, ABI-stable interfaces, compilation barriers, task-queues, runtime lifts for double-dispatch, and the list goes on and on and on and ...

### § 4.1 What does this give us that we don't have yet

#### § 4.1.1 Resolving overload sets for callbacks without lambdas

```
// generic context
std::sort(v.begin(), v.end(), [](auto const& x, auto const& y) {
    return my_comparator(x, y); // some overload set
});
```

becomes

```
// look ma, no lambda, no inlining, and less code generation!
std::sort(v.begin(),      v.end(),      declcall(my_comparator(v.front(),
    v.front())));
```

Note also, that in the case of a `vector<int>`, the ABI for the comparator is likely to take those by value, which means we get a better calling convention.

`static_cast<bool*>(int, int)>(my_comparator)` is not good enough here - the resolved comparator could take longs, for instance.

#### § 4.1.2 Copy-elision in callbacks

We cannot correctly forward immovable type construction through forwarding function.

Example:

```
int f(nonmovable) { /* ... */ }
struct {
    // doesn't work
    static auto operator()(auto&& obj) const {
        return f(std::forward<decltype(obj)>(obj)); // 1
    }
    // would work if we also had replacement function / expression aliases
    static auto operator()(auto&& obj) const
```

```

        = declcall(f(std::forward<obj>(obj)));          // 2
    } some_customization_point_object;

    void continue_with_result(auto callback) {
        callback(nonmovable{read_something()});
    }

    void handler() {
        continue_with_result(declcall(f(nonmovable{}))); // works
        // (1) doesn't work, (2) works
        continue_with_result(some_customization_point_object);
    }

```

## § 4.2 That's not good enough to do all that work. What else?

Together with [P2826R0], the two papers constitute the ability to implement *expression-equivalent* in many important cases (not all, that's probably impossible).

[P2826R0] proposes a way for a function signature to participate in overload resolution and, if it wins, be replaced by some other function.

This facility is the key to *finding* that other function. The ability to preserve prvalue-ness is crucial to implementing quite a lot of the standard library customization points as mandated by the standard, without compiler help.

## § 5 Needed Guidance

- Do we want to punt the syntax to reflection, or is this basic enough to warrant this feature? (Knowing that reflection will need more work from the user to get the pointer value).
- Do we care that it only works on unevaluated operands? (With the `static_cast` fallback in run-time cases)

## § 6 References

[N4856] David Sankel. 2020-03-02. C++ Extensions for Reflection.

<https://wg21.link/n4856>

[P1240R1] Daveed Vandevoorde, Wyatt Childers, Andrew Sutton, Faisal Vali, Daveed Vandevoorde. 2019-10-08. Scalable Reflection in C++.

<https://wg21.link/p1240r1>

[P2087R0] Mihail Naydenov. 2020-01-12. Reflection Naming: fix reflexpr.

<https://wg21.link/p2087r0>

[P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.

<https://wg21.link/p2237r0>

[P2300R6] Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein



Lelbach. 2023-01-19. `std::execution`.

<https://wg21.link/p2300r6>

[P2320R0] Andrew Sutton, Wyatt Childers, Daveed Vandevoorde. 2021-02-15. The Syntax of Static Reflection.

<https://wg21.link/p2320r0>

[P2826R0] Gašper Ažman. Replacement functions.

<https://wg21.link/P2826R0>